

Vel Tech Group of Institutions

Name: GOPICHAND KAKIVAI

Email: vtu29748@veltech.edu.in

Roll no: vtu29748

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L3

VTU_DAA_Week 2_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

In a kingdom far away, two types of soup are prepared daily — Soup A and Soup B. The royal chef always starts with n milliliters of each soup. Each time a servant is served, one of the following four serving operations is chosen at random (each with equal probability of 0.25):

Serve 100 ml of Soup A and 0 ml of Soup B

Serve 75 ml of Soup A and 25 ml of Soup B

Serve 50 ml of Soup A and 50 ml of Soup B

Serve 25 ml of Soup A and 75 ml of Soup B

The serving continues until either or both soups become empty. If the required soup amount for an operation exceeds the available quantity,

serve as much as possible (up to what remains).

You are to compute the probability that Soup A becomes empty first, plus half the probability that both soups become empty at the same time.

Special Note:

Since all operations serve soup in multiples of 25 ml, you must first scale n down to units of 25 ml before recursive processing:

Example

Sample Input 1

$n = 50$

Sample Output 2

0.625

Explanation

If we choose the first two operations, A will become empty first.

For the third operation, A and B will become empty at the same time.

For the fourth operation, B will become empty first.

So the total probability of A becoming empty first plus half the probability that A and B become empty at the same time, is $0.25 * (1 + 1 + 0.5 + 0) = 0.625$.

Input Format

The input consists of an integer n , representing the total volume of the soup.

Output Format

The output prints the probability that soup A will be empty first, plus half the probability that A and B become empty at the same time.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 50

Output: 0.625

Answer

```
#include <stdio.h>
#include <string.h>

double dp[201][201];

double solve(int a, int b) {
    if (a <= 0 && b <= 0) return 0.5;
    if (a <= 0) return 1.0;
    if (b <= 0) return 0.0;
    if (dp[a][b] >= 0) return dp[a][b];

    dp[a][b] = 0.25 * (
        solve(a - 4, b) +
        solve(a - 3, b - 1) +
        solve(a - 2, b - 2) +
        solve(a - 1, b - 3)
    );

    return dp[a][b];
}

int main() {
    int n;
    scanf("%d", &n);

    int m = (n + 24) / 25;

    for (int i = 0; i < 201; i++)
        for (int j = 0; j < 201; j++)
            dp[i][j] = -1.0;

    double result = solve(m, m);

    printf("%.3f", result);

    return 0;
```

}

Status : Correct

Marks : 10/10

2. Problem Statement

In a unique team-building exercise, a company has decided to distribute a certain number of resources among its departments based on mathematical powers. The company has x resources that need to be distributed such that each department receives a different power of an integer. Specifically, each department i receives n resources, where n is a given power.

George, the resource manager, needs to find all possible ways to distribute exactly x resources using the powers of integers starting from 1. Each integer's power can only be used once.

Write a program to help George determine the number of ways to distribute x resources in this manner using a recursive manner.

Example 1

Input:

10

2

Output:

1

Explanation:

$x = 10$

$n = 2$

$10 = 1^2 + 3^2$, hence we have only 1 possibility.

Example 2

Input:

100

2

Output:

3

Explanation:

$x = 100$

$n = 2$

$100 = 10^2$ OR $6^2 + 8^2$ OR $1^2 + 3^2 + 4^2 + 5^2 + 7^2$, hence total 3 possibilities.

Input Format

The first line of input consists of the integer x , representing the total number of resources.

The second line consists of the integer n , representing the power to which the integers are raised.

Output Format

The output should be a single integer representing the number of ways to distribute exactly x resources.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 100

2

Output: 3

Answer

```
#include <stdio.h>
```

```
#include <math.h>
```

```
int countWays(int x, int n, int num) {
```

```

    int val = pow(num, n);
    if (val == x) return 1;
    if (val > x) return 0;

    return countWays(x - val, n, num + 1) + countWays(x, n, num + 1);
}

int main() {
    int x, n;
    scanf("%d", &x);
    scanf("%d", &n);

    printf("%d", countWays(x, n, 1));
    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

In a small research lab, a programmer named Arin is analyzing binary data to detect system flags stored as set bits in an integer. One day Arin receives an integer and must determine how many bits are set to 1 in its binary representation.

Write a program to ensure that this task is done so Arin can continue validating the system's binary logs. The objective is to count all set bits in the given number using any valid approach.

Function specification: countSetBits(int n)

Input Format

The input consists of a single positive integer n.

Output Format

The output prints a single integer representing the count of set bits (1s) in the binary representation of n.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 21

Output: 3

Answer

```
#include <stdio.h>
```

```
int countSetBits(int n) {  
    int count = 0;  
    while (n) {  
        count += n & 1;  
        n >>= 1;  
    }  
    return count;  
}
```

```
int main() {  
    int n;  
    scanf("%d", &n);  
    printf("%d", countSetBits(n));  
    return 0;  
}
```

Status : Correct

Marks : 10/10

4. Problem Statement

Given a square matrix of size $n \times n$, rotate the elements along each layer (or boundary) of the matrix by one position in a clockwise direction. The rotation should be performed layer by layer, starting from the outermost layer and proceeding inward.

Examples:

Input 2:

3 3

1 2 3

4 5 6

7 8 9

Output 2:

4 1 2

7 5 3

8 9 6

Explanation:

For a clockwise boundary rotation by one step:

We start with the outermost layer (there's only one layer in a 3×3 matrix)

We save the top-left element (1)

Move left edge upward: 4 1, 7 4

Move bottom edge leftward: 8 7, 9 8

Move right edge downward: 6 9, 3 6

Move top edge rightward: 2 3

Place the saved element (1) in the second position of the top row, so 1 2

Input Format

The first line contains two integers R and C — the number of rows and columns in the matrix.

The next R lines each contain C integers, representing the elements of the matrix.

Output Format

The output displays the rotated matrix after performing the clockwise boundary rotation.

Refer to the sample input and output for formatting specifications.

Sample Test Case

Input: 4 4

1 2 3 4

5 6 7 8

9 10 11 12

13 14 15 16

Output: 5 1 2 3

9 10 6 4

13 11 7 8

14 15 16 12

Answer

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, m;
```

```
    scanf("%d %d", &n, &m);
```

```
    int a[5][5];
```

```
    for (int i = 0; i < n; i++)
```

```
        for (int j = 0; j < m; j++)
```

```
            scanf("%d", &a[i][j]);
```

```
    int top = 0, bottom = n - 1, left = 0, right = m - 1;
```

```
    while (top < bottom && left < right) {
```

```
        int prev = a[top + 1][left];
```

```
        for (int i = left; i <= right; i++) {
```

```
            int temp = a[top][i];
```

```
            a[top][i] = prev;
```

```
            prev = temp;
```

```
        }
```

```
        top++;
```

```
        for (int i = top; i <= bottom; i++) {
```

```
            int temp = a[i][right];
```

```
            a[i][right] = prev;
```

```

    prev = temp;
}
right--;

for (int i = right; i >= left; i--) {
    int temp = a[bottom][i];
    a[bottom][i] = prev;
    prev = temp;
}
bottom--;

for (int i = bottom; i >= top; i--) {
    int temp = a[i][left];
    a[i][left] = prev;
    prev = temp;
}
left++;
}

for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++)
        printf("%d ", a[i][j]);
    printf("\n");
}

return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

Given a 2D matrix of size ROW × COL, print all elements of the matrix in diagonal order.

Diagonal Order Definition:

Start from the first element at position (0,0). Traverse all diagonals starting from the leftmost column going downward, then continue with diagonals starting from the bottom row going rightward. Within each diagonal, print elements from the top-right corner to bottom-left corner.

Example:

Input matrix

5 4

1 2 3 4

5 6 7 8

9 10 11 12

13 14 15 16

17 18 19 20

Diagonal printing of the above matrix is

1

5 2

9 6 3

13 10 7 4

17 14 11 8

18 15 12

19 16

20

Input Format

The first line of input contains two integers, ROW and COL.

The next ROW lines each contain COL space-separated integers

Output Format

The output consists of multiple lines, one for each diagonal. Each line contains the matrix elements of that diagonal in the order from top-right to bottom-left, separated by spaces.

Refer to the sample input and output for formatting specifications.

Sample Test Case

Input: 5 4

1 2 3 4

5 6 7 8

9 10 11 12

13 14 15 16

17 18 19 20

Output: 1

5 2

9 6 3

13 10 7 4

17 14 11 8

18 15 12

19 16

20

Answer

```
#include <stdio.h>
```

```
int main() {
```

```
    int r, c;
```

```
    scanf("%d %d", &r, &c);
```

```
    int a[10][10];
```

```
    for (int i = 0; i < r; i++)
```

```
        for (int j = 0; j < c; j++)
```

```
            scanf("%d", &a[i][j]);
```

```
    for (int i = 0; i < r; i++) {
```

```
        int row = i, col = 0;
```

```
        while (row >= 0 && col < c) {
```

```
            printf("%d ", a[row][col]);
```

```
            row--;
```

```
            col++;
```

```
        }
```

```
        printf("\n");
```

```
    }
```

```
    for (int j = 1; j < c; j++) {
```

```
int row = r - 1, col = j;
while (row >= 0 && col < c) {
    printf("%d ", a[row][col]);
    row--;
    col++;
}
printf("\n");
}

return 0;
}
```

Status : Correct

Marks : 10/10